

Manual for Benn Venn's SD Loader Card reader for the VZ/Laser

V3.3 (Jan 2026)

Give your VZ200/VZ300 or equivalent a new lease on life with this SD card loader.

Load .VZ snapshots instantly onto your system, supports both Binary and Basic files.

Includes 128kbytes of onboard sram to max out your VZ system.

Index

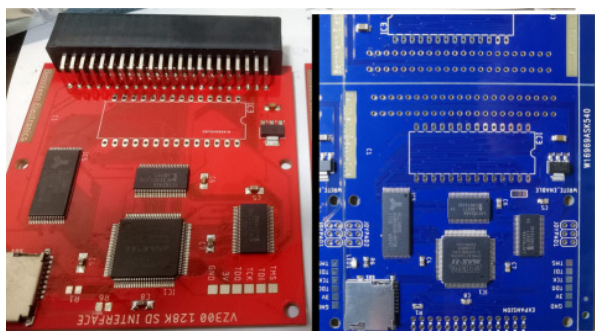
- Intro
- Overview of the SD Card reader 'SD Loader' BIOS Operating System.
- Command Line Interface.
- Formatting the memory card.
- Commands:
 - HELP.
 - DIR - directory Listing.
 - CD - change directory.
 - LOAD - loading Files.
 - SAVE - saving Files.
 - JOY - enables the loaders joysticks I/O port.
 - NOJOY - disables the loaders joysticks I/O port.
- SD Card reader I/O Ports.
 - SNES Joy pads.
- VZDOS routines, video and music fiddlings.
- About : The .VZ snapshot file.
- VZ Video – Notes from Ben.
- Resources.

Intro

In Aug 2019, Ben Grimmatt, an avid 8-bit enthusiast and entrepreneur of all Gameboy Z80 8-bit things, along with being a previous VZ owner, announced to the Facebook VZ & Laser group that he would create a SD card reader, with its own operating system, for these computers. All of the VZ & Laser community jumped with joy to hear such a thing. Eight weeks later his beta SD card reader was in operation. 50 were initially produced and went up for sale, and, within days just about all of them were sold out. Moving forward through Covid and world wide lockdowns to around August 2022, Mark II release of the SD card reader was ready for production, and another forty were up for sale. With an updated and bug-fixed operating system, version 2.0 is back better than before. Jan2026 and a third batch are now currently available.

<https://bennvenn.myshopify.com/products/vz300-sd-loader>

With it also being designed with an 8-bit I/O port along with a port for Gameboy gamepad joystick things, it comes with 128KB of ram that is accessible via bank switched memory. Great for storing graphics screens for animation.



The old and new versions of the SD Loader.



Benn Venn Electronics SD Loader, encased in a black plastic housing, printed and supplied by Igor Kromin.

Overview of the SD Card reader Operating System

A number of new commands are offered with the SD Card loader/reader.

Loading of BASIC and Binary files, and saving BASIC files are possible.

128 KB of memory is offered via bank switching, split into two pages, mapped from 8000-FFFF.

An I/O port on the reader itself for Gameboy gamepad type joypads with plastic 'knockouts' around the controller ports and the expansion port within the plastic case. They can be snapped out if you wish to install controllers or the expansion header, or left in there for the stock look.

The Laser200, Saloria Fellow, Texet 8000A, early Laser210 and very early German/Sanyo VZ200's that came only with 2k user RAM with 2k Video RAM have a 4k gap within the addressing when used with the SD Loader without modification. As such total memory available will still only be 2k. Your SD Loader for the use on these computers will require a different firmware, available from Benn Venn Electronics. The Loader was originally designed for a 6K user-ram VZ/Laser computers, and thusly the address gap will exist with these specific models. The firmware patch for the SD Loader for the use with these computers was released on the Facebook site under the files section on the 19th Nov 2022. Search for the name 'VZ300.POF' and it should be found. Once uploaded in to the Loader, this replaces the SD Loader's ram from 0x8000 to 0xFFFF, assuming only 2k of user ram is fitted above the 2k of video ram. It needs to be programmed with Altera/Intel USB Blaster. Talk directly to Ben should you need more information on this.

Plugging in the SD Loader, and typing the following, should show you your new total amount of RAM available to you in DOS BASIC. Note this does not indicate the extended RAM that is still available through the use of Bank switching.

Remove the SD Loader, and type this: `PRINT PEEK(30897)+256*PEEK(30898)`

Record the number. Then plug the SD Loader back in, and type it again. The number should have changed, indicating that you now have more user RAM memory.

The SD loader expands the 6kb 200/210 and the 300/310 out to 34kb of usable RAM (excluding video RAM). The same as if you had a factory VZ/Laser 64k RAM pack plugged in. Where is all of the other memory you ask? It is all tied up in bank switching, meaning that the 64k RAM pack has three more banks of RAM that can be selected and switched in & out at will. The SD Loader has seven additional banks of RAM – each bank being 16k in size.

The disappointment to all of this? : Only one chunk or bank of 16kb of all of this extra extended RAM goodness can only be accessed at any one time. It can be available from BASIC once top of memory and the stack pointer are moved down in memory, but is far easier to manipulate bank switching in assembly through port 112.

The card reader BIOS O/S firmware was designed around a SDHC class memory card being present. This means 2Gb or greater. 256,512meg & 1 gig cards wont work. The O/S uses the FAT32 file system which allows for up to a single 2Tb partition, however, Windows typically can only format up to a single 32Gb partition. Use third party partition tools to create a single 64 gig partition on a 64gig SD card.

Be aware when formatting your SD Card that the SD Loader project is expecting a MBR FAT32 format, and not a GUID FAT32 format. The difference in wording is subtle, but the physical format of the records, sectors, etc are vastly different. You may require a hex editor to wipe out the first few sectors on the SD Card in order to format the card successfully with a MBR FAT32 format.

-The saving of a BASIC file is limited to eight (8) characters within the filename.

-The directory listing has support for long filenames (greater than 8 characters), being displayed as '>'

-Sub directories are shown in the VZ/Laser's inverse colour.

-A small HELP screen is available from BASIC.

The SD new operating system commands are :

HELP	- shows a simple help page of commands.
DIR	- lists the files on the memory card.
CD"filename"	- Moves into the sub-directory named "filename"
LOAD"filename"	- Loads the program named "filename".
SAVE"filename"	- Saves the current program in memory as "filename"
JOY	- Enables the loaders joysticks I/O port. Disables normal VZ joysticks.
NOJOY	- Disables the loaders joysticks I/O port. Enables normal VZ joysticks.

CLI - Command Line interface

During the boot process, pressing <SPACE> will allow you into the command line interface. A few extra commands are added to within the CLI, listed below, which are mostly for memory manipulation - moving and writing commands. These are a simple way to modify or enter data into memory. Type HELP for a list of commands.

Note, Ben has included key anti-bounce within his source, however, some key bouncing may still occur. Type slowly and carefully.

Commands within the CLI are :

HEXVIEW address(16)	: View memory locations.
WRITE destination:data	: Write directly to a memory location.
COPY src:dest:count	: Copy from scr to dest. an amount of memory.
FILL destination:val:count	: Fill an amount of memory with a constant.
JUMP address(16)	: Jump to an address in memory.
IN address(8)	: Read a byte value in from a port I/O address.
OUT address(8):data	: Write a byte value to a port I/O address.
BASIC	: Boot to BASIC
DIR	: Perform a directory listing.
LOAD filename	: Load in a file named filename.
RUN	: RUN a program in memory.
CLS	: Clear the screen.

Formatting the memory card

Take a 2 to 64 gig micro SD card, (Tested: 64GB SDXC) Plug it into an adapter, plug the adapter into your favourite operating system on your home computer, and format away the card. MBR FAT32. Not FAT or exFAT or NTFS.

Preference for formatting is to use a Windows operating system to format the memory card.

Both Linux and OSX FAT32 formats can be a little off, (clearly for this to be a known factor, these formats are not 100% compatible with Windows). Perhaps the newer versions of these Operating Systems use the newer GUID properties rather than old-school MBR. Unexpected results can potentially happen when reading/writing to the memory card, and corruption to both data and filenames, and even data loss is quite possible. For 100% confidence in things always working, always format the SD Card within a known operating system that can guarantee a MBR FAT32 format. Eg, 200% confidence is guaranteed when formatting is done by a clunky old Windows 7 system.

A note from Ben mentions that cluster size is important, even though it can handle clusters from 512bytes to 64kbytes. If the Cluster is larger than the file size, the O/S doesn't need to link chains.

I personally have found that the cluster size of 16KB has worked fine on a 2Gig and 64Gig SD cards, having stored well over 200 .VZ snapshot files in tens of sub-directories. Always nicely eject the SD card, otherwise data loss is quite possible if files in cache are still yet to be written. An ejection forces the writing and the closure of any files still in cache memory.

Upon power up the SD card will be initialised, FAT32 will be initialised. If VZDOS.VZ is found on the root directory, it will be copied to \$4000 and executed. This allows the latest firmwares to be launched without reflashing the EPROM. BASIC is loaded by default. This is for both Mk I and Mk II Loaders.

From the Help : "IF VZDOS.VZ is found on the SD Card this new version of the BIOS will be loaded in place of the default BIOS."

Mark I SD Card reader loaders, find, download, and copy over the v2.0 operating system file 'VZDOS.VZ' on to your memory card to upgrade this particular SD Card reader to the latest version.

Mark II of the SD card readers were supplied with v2.0 of the firmware BIOS operating system. At the time of editing, v2.32 of 'VZDOS.VZ' is now available and should be copied to the root of your SD memory card.

On the Loader PCB there is a jumper which enables EEPROM re-writing which allows you to update the firmware version via CLOADing, and this will directly to the EEPROM. Once finished, the jumper can be removed to disable re-writing.

VZDOS.VZ : BIOS firmware for SD memory card.

VZDOS.BIN : BIOS firmware for EPROM/EEPROM programming.

VZDOS.OBJ : BIOS firmware for EPROM/EEPROM programming. (Same file as above .BIN)

VZDOS.WAV : BIOS firmware for cassette CLOAD'ing to the EEPROM once the jumper is set.

These files can be found within the files section of the VZ/ Laser Fans Facebook group.

For SD card readers that refuse to read memory cards correctly, try this fix:

Format Ultra 2Gb, 8Gb, 16Gb, 32Gb, 64gb micro sd card.

Use hex program "HxD". (<https://mh-nexus.de/en/hxd/>)

Load sd card physical drive

Go to top / offset 0.

zero out the entire Sector 0 of the physical memory card disk to wipe out the partitions.

Save drive.

Format normally. FAT32. select Default sector allocation.

Should work fine using SD loader DOS v2.32 "VZDOS.VZ"

An interesting feature to experiment with is the further use of the auto-boot 'VZDOS.VZ' file as an autostart program, which could be a selection menu, a game, or an auto boot graphics and music demonstration program. This is saved in a separate directory within the SD ROM files.

Operating System Commands

HELP

From version 2.0, typing in 'HELP' will give you a one/two page screen showing hints and tips.

Directory Listing

Type DIR to show the contents on the memory card. This will show 16 files and then stop advising you to press space to continue or press break to end the listing.

The directory listing will show long file names as > .

This is when a file is saved from Windows with a filename greater than eight characters.

Loading Files

On your home computer, scrounge the internet and download all of the .VZ snapshot files that you can find. Copy these over on to your SD memory card. Take the card and plug it into your VZ Card reader.

Perform a directory listing to see your files.

To load a particular basic or binary .VZ file, simply type :

LOAD "VZ FILE"

Note that the FAT32 system only allows for eight characters for the filename.

The operating system will detect if the file is basic (F0) or binary (F1). If it is a binary file, it will attempt to automatically execute the file. If it is a basic file, the file will load, but will leave you at DOS BASIC, allowing you to LIST or RUN the program, or CSAVE to tape as an example.

CD – change directory

The command CD will allow you to change from one directory to another.

CD"directory name" : will change from the current directory into the directory named "directory name"

CD"..": will change from the current directory, up one directory level.

Saving Files

SAVE"filename" will allow you to save BASIC programs in memory to the SD memory card.

Note that the file system only allows for a maximum of eight characters within the filename.

If you attempt to use nine or greater, a syntax error will occur.

If you attempt to save a file with the same filename that already exists on the memory card, the operating system will ask / confirm if you wish to overwrite. Nice touch Ben.

JOY

The JOY command will enable the SD card reader/loader joystick interface that is built into the reader.

This will disable the normal VZ/Laser joysticks and its interface.

NOJOY

The NOJOY command will disable the SD loader joystick interface to allow the use of normal VZ joysticks that are attached to their own VZ/Laser interface.

Note: Sub directories need to be created on to the memory card from within the Windows operating system, (ie CMD → MD FILENAME). Or within Linux /OSX if this is working for you. (not recommended)

SD Card reader I/O Ports

- 55 RAM map. Set to 0 for VZ200 compatibility (RAM mapped from \$9000-\$FFFF)
 Set to 1 for VZ300 compatibility (RAM mapped from \$B800-\$FFFF)
 OUT 55, 8 Turns on the LED for testing. Led is controlled by bit 3.
 OUT 55, 0 Turns off the LED for testing.
- 56 SPI port CFG for SD interface.
- 57 SPI port Data for SD interface.
- 58 Bit0 = EPROM(0) or SRAM(1) mapped to DOS ROM AREA (Note, SRAM write is enabled even when EPROM is selected. This means you can code a loop to read and write from the DOS ROM area to copy itself into ram before switching to SRAM)
 Bit1 = SRAM BankLow - DOS ROM SRAM bank 0 or 1 (\$4000-\$67FF)
 Bit2 = SRAM BankHigh - SRAM Bank 0 or 1 (Expanded RAM up to \$FFFF)
 Bit3 = on board LED if fitted. (If you mod your VZ's ROM /CS line, you can use this signal to disable the original ROM and map in SRAM allowing you to load BASIC rom's from SD card. This has not been implemented yet.
- 74 Control the SD Loader's Joypad SNES controlling interface. Enabled by default.
 OUT74,0 : Disable the SNES interface. Enables the VZ joysticks on the OEM interface.
 OUT74,1 : Enable the SNES controller interface. Disables the OEM VZ Joystick interface

SNES Joypads

SNES pad IO addresses - The original VZ addresses are supported as well as the expanded joypad address space which gives you access to all 12 buttons on the SNES pad.

SNES pad I/O addresses. (All 12 buttons on the SNED pad are supported)

```
constant Cont1a      : integer := 46;  
constant Cont1b      : integer := 45;  
constant Cont2a      : integer := 43;  
constant Cont2b      : integer := 39;  
constant ContBa      : integer := 42;  
constant ContBb      : integer := 37;  
constant Joy1a       : integer := 64;  
constant Joy1b       : integer := 65;  
constant Joy2a       : integer := 66;  
constant Joy2b       : integer := 67;  
constant DDR0add     : integer := 68;  
constant DDR1add     : integer := 69;  
constant DDR2add     : integer := 70;  
constant IO0add      : integer := 71;  
constant IO1add      : integer := 72;  
constant IO2add      : integer := 73;  
constant JoyDisablePort : integer := 74;
```

-The Cont ports are as the VZ hardware engineers intended, port values are decimal.

-JoyXY ports are 2 bytes each controller to map the full 12 bits of the SNES pad or 8 bits of the NES pad.

-DDR_x is the IO port direction register, 0=output, 1=input for each pin.

-IO_x is the Input/output port for each 8 bit IO port. You can read back the port even if it is set to an output to check the current state of the pin.

To check if the VZ/Laser joysticks or SNES pads are working, this simply program will show the input figures.

```
10 PRINT"LEFT: ";INP(43);INP(39);"RIGHT: ";INP(46);INP(45)  
20 GOTO 10
```

VZDOS routines, video and music fiddlings.

\$5F00 - OpenFile

Description : opens selected file
In : reg HL = pointer to filename. Reg A = filehandle number. Starting typically at 1.
Call : call 5f00h
Jump table : 5ff0h

\$5F03 - Select Handle

Description : Selects Handle. Loads first frame into the ram buffer.
In : a = filehandle number. Starting typically at 1.
Call : call 5f03h

\$5F06 - DrawFrame

Description : Draw frame from buffer to video ram.
Call : call 5f06h
Jump table : 5FF6h

\$5F09 - FAT NextSector

Description : Performs a sector read from the SD Card, then performs a fast copy of 512 bytes
: from 'ram buffer' to a destination memory location that is configurable (\$FB80)
In : Word location at \$FB80 is a "destination memory" location for the copy.
: Example : Setting \$FB80 to \$7000, copies a sector of 512 bytes over from the
: rambuffer to the value set at FB80, in this case, video screen (which is at \$7000).
: This will copy a MODE(0) screen, or 512 bytes (one quarter) of MODE(1) screen.
: Second example, setting \$FB80 to \$B000, then calling \$5F09, will copy 512
: bytes from the RAM buffer, which all has its own internal pointers, to \$B000,
: where you are then free to manipulate or further read from this location.
: This feature is used in the continuous music examples that have been released.
Call : Call \$5F09

\$5F15 - SEEK

Description : File Seek.
Call : Call \$5F15.
In : 0=go back to the start of the file. Any other number will skip further into the file.
: calling seek twice with DE=10, will skip 20 sectors etc.
: This will let you seek up to 32mbytes into a DAT file per call.
: It could be a little faster but overall, it's not too bad. Maybe 1.5 seconds to seek
: 32mbytes into a file

\$5FF0 - Jump table

\$5FF0 : OpenFile
\$5FF3 : SD2VRAM
\$5FF6 : DrawFrame
\$5FF9 : Fat32_Root
? unsure if \$5F15 has been added. : File Seek.

Example of video, taken from Ben's "AHAFast" video.

"Take On Me", by Aha! is the groups unofficial music video by default. (...just go with it.)

Due to the auto loading of the first sector before the loop, we can start at \$7200 instead of \$7000, otherwise everything is 'out-of-whack' by 512bytes, and the video file would need to be further edited.

```

SdbufferAddress equ 0FB80h
FAT_NextSector   equ 05F09h

org      7FE8h
db       20h, 20h, 00h, 00h, 56h, 5Ah, 44h, 4Fh ; generic header
db       53h, 00h, 00h, 00h, 00h, 00h, 00h, 00h
db       00h, 00h, 00h, 00h, 00h, 0F1h, 00h, 80h

org      8000h

ld       a, 08H
ld       (6800h), a           ; set hires mode
ld       hl, FILENAME
ld       a, 1
call    $5f00                ; openFile
ld       a, 1
call    $5f03                ; selectHandle - This loads the first frame
                                   ; into RAMbuffer, not VRAM! Add an extra sector
                                   ; to the DAT file, or copy from buffer to vram

infloop: ld       HL, 7200h
ld       (SdbufferAddress), HL
call    FAT_NextSector        ; 512 bytes to vram
ld       HL, 7400h
ld       (SdbufferAddress), HL
call    FAT_NextSector        ; 512 bytes to vram
ld       HL, 7600h
ld       (SdbufferAddress), HL
call    FAT_NextSector        ; 512 bytes to vram
ld       HL, 7800h
ld       (SdbufferAddress), HL
call    FAT_NextSector        ; 512 bytes to vram
ld       HL, 7A00h
ld       (SdbufferAddress), HL
call    FAT_NextSector        ; 512 bytes to vram
jr      infloop

FILENAME: db       22h, "AHA", 22h           ; filename to load
end

```

...and then you need to encode a video (eg, AHA) using the below technique over the page.

About : The .VZ snapshot file

The .VZ format was put together rather very quickly one night in 1999 by Brian Murray to get "something / anything" to work with Guy Thomason's upcoming new emulator to be able to load and save files. The format is a memory "snapshot" and not necessarily technically correct by any means. It is a legitimate true VZ enthusiast hacked-together format, that became the standard as it was the first slapped-together data format outside of the actual physical hardware. Purists should hate this format.

.CAS : This cassette file format came a little later but we may as say that noone uses.

.CVZ : Gavin Turner created his own .CVZ image for his DSVZ200 and WINVZ300 emulators.

The ".VZ" file format consists of a 24 byte header, and then followed by the program data.

typedef struct vzFile	<u>Header in Assembly:</u>	
{	byte vzmagic[4];	defs 'VZF1' ; vzmagic [04].
	byte filename[17];	defs 'FILENAME12345678' ; filename [17].
	byte ftype;	defb \$F1 ; Filetype [01].
	byte start_addrl;	defw \$8000 ; Start address [02].
	byte start_addrh;	
}	VZFILE;	

F0 being for a BASIC program with a starting address typically at 7AE9.

F1 being a binary, typically being an autostart/auto execute game or utility.

VZ Video – Notes directly from Ben.

You'll need:

Download FFMPEG

Download ImageMagik

To create the DAT file for the VZ: at the command line -

FFMPEG -i source.mp4 -r 12 -f image2 -vf format=gray -s 128x64 .\output\%d.bmp

r 12 = frame rate, 12 frames per second. Adjust to suit. This will be higher when I tidy up the code. Source can be a range of video formats. Make an output directory. After running this, you'll get hundreds/thousands of bitmap files in the output folder. You'll then want to convert the greyscale to 1bit format. At the command line in the output folder, type:

MAGICK mogrify -dither None -monochrome *.bmp

This will convert from greyscale to 1bit using an intensity threshold algorithm to set the black/white level. For a better image in certain video's, you can use dithering

MAGICK mogrify -dither FloydSteinberg -monochrome *.bmp

This will take a minute or two. You now need to convert the bitmaps to raw images and stack them. I've coded a cpp app to do this for me. Run it from the output directory.

ConsoleApplication1.exe

The source is included if you want to tinker. It is based on Jonnection's work (<https://www.instructables.com/Streaming-video-from-SD-card-to-Nokia-LCD-with-Ard/>)

The VZ side code:

Still a work in progress: The goal - You can open multiple files from the SD and attach a file handle to each, 0-15. You can then seek in the file or read 1 sector at a time from them (512 bytes). It'll be up to you how you use this data.

To display video:

Point HL to the video file name. With A holding the file handler number e.g.

ld a,0

ld HL,VideoFile

call OpenFile ;opens the file and assigns it to Handle 0

call SD2VRAM ; Copy one frame to VRAM from the file

loop:

call DrawFrame ; Draw another frame from the file

jr loop

VideoFile: .db 22H,"TAKEONME",22h

A jump table will be hardcoded into the VZDOS ROM. You will need to update your VZDOS ROM to make use of these changes, or include the functions in your game code.

Jump table:

.org 5FF0h

jp OpenFile

jp SD2VRAM

jp DrawFrame

jp Fat32_Root

Resources/References:

<https://www.facebook.com/groups/4609469943>
<https://bennvenn.myshopify.com/products/vz300-sd-loader>
<https://mh-nexus.de/en/hxd/>
<https://ffmpeg.org/>
<https://imagemagick.org/index.php>

VZ/Laser Fans Facebook Group.
Benn Venn loader shop listing.
Hex program "HxD".
FFMPEG
ImageMagick. (mindblowing stuff !)

Examples

What can the SD Loader be used for?
Videos, music, animation, games, all of your programs.

Here are a few real-world working examples.

<https://www.youtube.com/watch?v=YXWYmT9W4m0>
<https://www.youtube.com/watch?v=ouEL4OW3U6U>

Danger Mouse cartoon.
Money for Nothing.

...and Ben even ported the entire movie "Interstella" to a jazillion JPG files and placed them altogether. Yes, you too can now watch the entire Interstella movie in green and red on the VZ/Laser should you wish to. A number of clips are now available for watching that are found in the 'VZ Everything software' repository.

ROM Releases

V0.0 Nov 2019.

v1.5

v1.6

v1.7 Jan 2022. Overwrite support added, break and reuse cluster chains. Added new CLI commands (FILL, COPY). Partitioned SD's now supported. Directories Supported

v1.8 Jan 2022. Directory listing 15/16 lines issue bug fix. Now supports opening of Long file names.

v2.0 Oct 2022. Further small issues fixed, along with SNES port commands (JOY/NOJOY) for Mk II.

v2.1 Oct 2022. Fixes.

v2.2 Nov 2022. Optimised SD functions (Can dump a sector direct to VRAM)
Included is test code to show how to use it, and a 24 frame per second AHA dat file
At 24fps the video runs fast so we're probably closer to around 28-30 frames per second.
If used in GRAFX mode, that'll be divided by 3, but 10fps is not too bad!
It should at least remove the visible screen drawing.

v2.31 . 22 May 2023. Added a seek command. Calling Seek with the number of sectors to advance in DE.
0=go back to the start of the file. Any other number will skip further into the file. calling seek twice with DE=10, will skip 20 sectors etc. This will let you seek up to 32mbytes into a DAT file per call. It could be a little faster but overall, it's not too bad. Maybe 1.5 seconds to seek 32mbytes into a file. I might modify it later for an 32bit absolute sector in a file. The call is located at 0x5F15

v2.32 24/May/2023. Bug fix.

This manual was slapped together very crudely by Dave. All feedback & errors welcomed; send to either /dev/nul or bushy555@gmail.com (same difference really)